Core Java MCQ Practice Bank – CDAC PGCP-AC (Java 17)

200 medium-to-hard MCQs, original and aligned to your syllabus (Sessions 1–30). Tags: [Output] = predict output, [Scenario] = real-world situation, [Trap] = commonly confused, [Concept] = theory. Answer key with short reasoning is at the end of each section. Try first, then check.

Section 1: Java Basics, Data Types, Operators, Control, Arrays (Q1–Q22)

Q1. [Output]

```
byte b = 10;
b = b + 5;
System.out.println(b);
```

- A) 15
- B) Compile-time error
- C) Runtime ArithmeticException
- D) 5

Q2. [Output]

```
int x = 5;
int y = x++ + ++x;
System.out.println(x + " " + y);
```

- A) 6 11
- B) 7 12
- C) 7 11
- D) 6 12

Q3. [Trap]

```
System.out.println('a' + 1 + "b" + 'c');
```

- A) abc

- B) 98bc
- C) bbc
- D) a1bc

Q4. [Trap]

```
char c = 'A';
c += 2;
System.out.println(c);
```

- A) C
- B) 67
- C) Compile-time error
- D) A2

Q5. [Output]

```
int i = 10;
long l = i;
float f = l;
int j = (int) 3.99;
System.out.println(f + " " + j);
```

- A) 10 3
- B) 10.0 3
- C) 10.0 4
- D) Compile-time error

Q6. [Trap]

```
final int a = 10;
int c = 10;
byte b1 = a;   // line 1
byte b2 = c;   // line 2
```

- A) Both lines fail
- B) Only line 1 fails
- C) Only line 2 fails
- D) Both compile

Q7. [Output]

```
int x = Integer.MAX_VALUE;
x++;
System.out.println(x);
```

- A) 2147483648
- B) -2147483648
- C) ArithmeticException
- D) 0

Q8. [Output]

```
System.out.println(10/3 + 10%3 + 10.0/4);
```

- A) 6.5
- B) 6.83
- C) 7.0
- D) 5.5

Q9. [Trap]

```
System.out.println((-7 % 3) + (7 % -3));
```

- A) 0
- B) -2
- C) 2
- D) -1

Q10. [Output]

```
int n = 2;
switch (n) {
  case 1: System.out.print("A");
  case 2: System.out.print("B");
  case 3: System.out.print("C"); break;
  default: System.out.print("D");
}
```

- A) B
- B) BC
- C) BCD

- D) C

Q11. [Output]

```
int k = 4;
int res = switch (k) {
  case 1, 2 -> 10;
  case 3, 4 -> { int t = k * 2; yield t + 1; }
  default -> 0;
};
System.out.println(res);
```

- A) 8
- B) 9
- C) 10
- D) 0

Q12. [Concept] Which statement about switch expressions (Java 14+) is correct?

- A) A switch expression on an int must be exhaustive, so it needs a default (or cover all values)
- B) Arrow-form cases need a break statement
- C) Arrow-form cases fall through to the next case
- D) A switch expression cannot produce a value from a block

Q13. [Output]

```
int[] a = {1, 2, 3};
int[] b = a;
b[0] = 99;
System.out.println(a[0] + " " + (a == b));
```

- A) 1 true
- B) 99 true
- C) 99 false
- D) 1 false

Q14. [Trap]

```
int[][] m = new int[3][];
System.out.println(m[0]);
```

- A) 0
- B) null
- C) [I@1b6d3586 (some hash)
- D) NullPointerException

Q15. [Output] (Pseudo-code style) Find the value of `s`:

```
int[][] m = {{1,2,3},{4,5,6},{7,8,10}};
int s = 0;
for (int i = 0; i < 3; i++) s += m[i][i];
System.out.println(s);
```

- A) 15
- B) 16
- C) 17
- D) 22

Q16. [Concept] Which is a valid Java identifier?

- A) 2ndVar
- B) my-var
- C) _value
- D) class

Q17. [Trap] Which of the following is a reserved literal and NOT a keyword?

- A) goto
- B) null
- C) const
- D) strictfp

Q18. [Output]

```
int x = 0b101 + 010 + 0x10;
System.out.println(x);
```

- A) 26
- B) 29
- C) 31
- D) 23

Q19. [Trap]

```
System.out.println(0.1 + 0.2 == 0.3);
System.out.println(0.5 + 0.25 == 0.75);
```

- A) true true
- B) false true
- C) false false
- D) true false

Q20. [Output]

```
class T {
  int i; boolean b; String s; char c; double d;
  void p() { System.out.println(i + " " + b + " " + s + " " + (int) c + " " +
d); }
}
new T().p();
```

- A) 0 false null 0 0.0
- B) 0 false (empty) 0 0.0
- C) Compile-time error (uninitialized variables)
- D) null false null 0 0

Q21. [Concept] Which statement about the JVM is correct?

- A) The JRE contains the javac compiler
- B) The JIT compiler is part of the JVM and compiles frequently used bytecode to native code at runtime
- C) Metaspace is a part of the Java heap
- D) The class loader subsystem executes the bytecode

Q22. [Scenario] A student saves `public class Test { ... }` in a file named `Demo.java` and compiles it. What happens?

- A) Compiles; run with `java Demo`
- B) Compile-time error: public class Test must be declared in Test.java
- C) Compiles but fails at runtime
- D) Compiles; run with `java Test`

Answer Key – Section 1

1. **B** – `b + 5` is an int; assigning to byte needs a cast (lossy conversion).
2. **B** – x++ gives 5 (x=6), ++x makes x=7 and gives 7; y = 12, x = 7.
3. **B** – 'a'+1 is int 98; then string concatenation gives "98b", then "98bc".
4. **A** – Compound assignment (+=) has an implicit cast, so c becomes 'C'.
5. **B** – f = 10.0 (float), and (int)3.99 truncates to 3.
6. **C** – A final variable with a compile-time constant that fits in byte can be narrowed implicitly; the non-final c cannot.
7. **B** – Integer overflow wraps around silently.
8. **A** – 3 + 1 + 2.5 = 6.5.
9. **A** – Result of % takes the sign of the dividend: -1 + 1 = 0.
10. **B** – Falls through from case 2 to case 3 until break: BC.
11. **B** – t = 8, yield 8 + 1 = 9.
12. **A** – Switch expressions must be exhaustive; arrow cases never fall through and need no break.
13. **B** – Arrays are objects; b and a refer to the same array.
14. **B** – Rows of a jagged array default to null; printing null prints "null".
15. **B** – 1 + 5 + 10 = 16.
16. **C** – Identifiers can start with letter, _ or $; they cannot be keywords or contain '-'.
17. **B** – null (like true and false) is a literal; goto, const are reserved keywords.
18. **B** – 5 + 8 (octal 010) + 16 = 29.
19. **B** – 0.1+0.2 is not exactly 0.3 in binary floating point; 0.5 and 0.25 are exactly representable.
20. **A** – Instance variables get default values; (int)'\u0000' = 0.
21. **B** – JRE does not include javac; Metaspace is native memory; the execution engine runs bytecode.
22. **B** – A public top-level class must live in a file with the same name.

Section 2: OOP, Static, References, Constructors, Memory, Records (Q23–Q40)

Q23. [Output]

```
class Counter {
  static int s = 0; int i = 0;
  Counter() { s++; i++; }
}
Counter a = new Counter(), b = new Counter(), c = new Counter();
System.out.println(a.s + " " + a.i + " " + c.i);
```

- A) 3 3 3
- B) 3 1 1
- C) 1 1 1
- D) 3 3 1

Q24. [Scenario] A Student class has `String name` (instance) and a static method `showCollege()` that prints `name`. What happens?

- A) Prints null
- B) Prints the college name
- C) Compile-time error: non-static variable cannot be referenced from a static context
- D) NullPointerException at runtime

Q25. [Output]

```
static void change(int[] arr, int n) {
  arr[0] = 50; n = 50; arr = new int[]{9, 9};
}
// main
int[] a = {1, 2}; int n = 1;
change(a, n);
System.out.println(a[0] + " " + n + " " + a.length);
```

- A) 1 1 2
- B) 50 50 2
- C) 50 1 2
- D) 9 1 2

Q26. [Trap]

```
static void swap(Student s1, Student s2) { Student t = s1; s1 = s2; s2 = t; }
// main
Student a = new Student("Ann"), b = new Student("Ben");
swap(a, b);
System.out.println(a.name + " " + b.name);
```

- A) Ben Ann
- B) Ann Ben
- C) Compile-time error
- D) NullPointerException

Q27. [Output]

```
class Box { int v = 1; }
static void m(Box b) { b.v = 5; b = new Box(); b.v = 9; }
// main
Box x = new Box(); m(x);
System.out.println(x.v);
```

- A) 1
- B) 5
- C) 9
- D) Compile-time error

Q28. [Output]

```
class A {
  A()      { this(5); System.out.print("A0 "); }
  A(int x) { System.out.print("A1 "); }
}
new A();
```

- A) A0 A1
- B) A1 A0
- C) A1
- D) Compile-time error

Q29. [Trap]

```
class Emp { Emp(String n) { } }
Emp e = new Emp();
```

- A) Compiles; name is null
- B) Compile-time error: no suitable constructor
- C) Runtime error
- D) Default constructor is still generated

Q30. [Output]

```
class T {
  static { System.out.print("S "); }
  { System.out.print("I "); }
  T() { System.out.print("C "); }
  public static void main(String[] a) { new T(); new T(); }
}
```

- A) S I C I C
- B) I C S I C C
- C) S I C
- D) I C I C

Q31. [Concept] For `Student s = new Student();` declared inside a method, which is correct?

- A) Both s and the object are on the stack
- B) s is on the heap, object on the stack
- C) s (the reference) is on the stack, the object is on the heap
- D) Both are always on the heap

Q32. [Scenario] A recursive factorial method has no base case. What will most likely happen when it is called?

- A) OutOfMemoryError
- B) StackOverflowError
- C) StackOverflowException (checked)
- D) Compiler warning only, returns 0

Q33. [Trap]

```
class S { static int v = 10; }
S s = null;
System.out.println(s.v);
```

- A) NullPointerException
- B) 10
- C) null
- D) Compile-time error

Q34. [Output]

```
class P { int id; P(int i) { id = i; } }
P p1 = new P(1), p2 = new P(1);
System.out.println(p1 == p2);
System.out.println(p1.equals(p2));
```

- A) true true
- B) true false
- C) false false
- D) false true

Q35. [Output]

```
record Point(int x, int y) { }
System.out.println(new Point(1, 2));
```

- A) Point@1b6d3586
- B) Point[x=1, y=2]
- C) Point(1,2)
- D) Compile-time error

Q36. [Concept] Which statement about records is TRUE?

- A) Record fields can be changed using auto-generated setters
- B) A record can extend another class
- C) A record can implement interfaces and declare static methods
- D) A record's fields are non-final by default

Q37. [Trap] Given `record Student(int roll, String name) {}`, how do you read name from another class?

- A) s.getName()
- B) s.name()
- C) s.name
- D) s.get("name")

Q38. [Concept] Best description of the difference between a static variable and a reference variable:

- A) A static variable has one copy per class shared by all objects; a reference variable holds a reference to an object (and may itself be static or not)
- B) They are the same thing
- C) Reference variables are shared by all objects of a class
- D) Static variables can only be of primitive type

Q39. [Scenario]

```
class BankAccount { static String bankName = "ABC"; }
BankAccount a1 = new BankAccount(), a2 = new BankAccount();
a1.bankName = "XYZ";
System.out.println(a2.bankName);
```

- A) ABC
- B) XYZ
- C) null
- D) Compile-time error

Q40. [Trap]

```
Student() {
  System.out.println("x");
  this(5);
}
```

- A) Prints x, then continues normally
- B) Compile-time error: call to this() must be the first statement
- C) Runtime error
- D) Infinite recursion

Answer Key – Section 2

23. **B** – static s is shared (3); instance i is per object (1).
24. **C** – Static methods have no `this`, so they cannot access instance members directly.
25. **C** – Array contents change through the reference; n is a copy; reassigning arr affects only the local copy.
26. **B** – Java is always pass-by-value; the references are copied, so swapping copies does nothing.
27. **B** – b.v = 5 changes the shared object; then b points to a new Box locally.
28. **B** – this(5) runs A(int) first, then the rest of A().
29. **B** – Defining any constructor removes the default one.
30. **A** – Static block runs once at class load; instance block runs before every constructor body.
31. **C** – Local reference on the stack, object on the heap.
32. **B** – Endless recursion exhausts the stack: StackOverflowError (an Error, not an Exception).
33. **B** – Static members are resolved by the declared type, so no NPE.
34. **C** – Default equals() is identity comparison, same as ==.
35. **B** – Records auto-generate a readable toString.
36. **C** – Records are final, fields are private final, they extend java.lang.Record, and can implement interfaces.
37. **B** – Accessors are named like the component: name().
38. **A** – Static means one per class; a reference variable only stores a reference.
39. **B** – Access through any reference changes the single shared static variable.
40. **B** – In Java 17, this(...)/super(...) must be the first statement.

Section 3: Inheritance, Polymorphism, Packages, Access Control, Casting (Q41–Q62)

Q41. [Trap]

```
class A { int x = 10; void show() { System.out.print("A "); } }
class B extends A { int x = 20; void show() { System.out.print("B "); } }
A obj = new B();
obj.show();
System.out.println(obj.x);
```

- A) B 20
- B) B 10
- C) A 10
- D) A 20

Q42. [Trap]

```
class P { static void hi() { System.out.print("P"); } }
class C extends P { static void hi() { System.out.print("C"); } }
P p = new C();
p.hi();
```

- A) P
- B) C
- C) Compile-time error
- D) Runtime error

Q43. [Output]

```
class A { A() { System.out.print("A "); } }
class B extends A { B() { System.out.print("B "); } }
class C extends B { C() { System.out.print("C "); } }
new C();
```

- A) C B A
- B) A B C
- C) C
- D) A C

Q44. [Trap]

```
class A { A() { show(); } void show() { System.out.print("A"); } }
class B extends A {
  int v = 5;
  void show() { System.out.print("v=" + v); }
}
new B();
```

- A) v=5
- B) v=0
- C) A
- D) NullPointerException

Q45. [Trap] Only these overloads exist: `f(long)`, `f(Integer)`, `f(Object)`. What does `f(5)` call?

- A) f(long)
- B) f(Integer)
- C) f(Object)
- D) Compile-time error

Q46. [Trap]

```
static void m(Object o) { System.out.print("O"); }
static void m(String s) { System.out.print("S"); }
m(null);
```

- A) O
- B) S
- C) Compile-time error: ambiguous
- D) NullPointerException

Q47. [Concept] Parent has `protected Number get() throws IOException`. Which override in a subclass is ILLEGAL?

- A) `public Integer get()`
- B) `protected Number get() throws FileNotFoundException`
- C) `protected Number get() throws Exception`
- D) `protected Integer get()`

Q48. [Trap]

```
class A { void f(A a) { System.out.print("A-A"); } }
class B extends A { void f(B b) { System.out.print("B-B"); } }
A obj = new B();
obj.f(new B());
```

- A) B-B
- B) A-A
- C) Compile-time error
- D) Runtime error

Q49. [Output]

```
class Animal { void sound() { } }
class Dog extends Animal { void sound() { System.out.print("Woof"); } void
fetch() { System.out.print("fetch"); } }
Animal a = new Dog();
a.fetch();
```

- A) fetch
- B) ClassCastException
- C) Compile-time error
- D) Woof

Q50. [Scenario] In an Employee system, `Animal a = new Animal(); Dog d = (Dog) a;` style code is written as `Person p = new Person(); Employee e = (Employee) p;`. What happens?

- A) Compile-time error
- B) Compiles; ClassCastException at runtime
- C) Works; e is null
- D) NullPointerException

Q51. [Output]

```
Object o = "Java";
if (o instanceof String s && s.length() > 3)
System.out.println(s.toUpperCase());
else System.out.println("no");
```

- A) JAVA

- B) Java
- C) no
- D) Compile-time error

Q52. [Trap]

```
String s = null;
System.out.println(s instanceof String);
```

- A) true
- B) false
- C) NullPointerException
- D) Compile-time error

Q53. [Scenario] Class `A` in package `p1` has `protected void m()`. Class `B extends A` in package `p2` contains: `A a = new A(); a.m();` and `this.m();`. Which is true?

- A) Both compile
- B) Only this.m() compiles
- C) Only a.m() compiles
- D) Neither compiles

Q54. [Concept] A member declared with no access modifier (default) is accessible from:

- A) Classes in the same package only
- B) Subclasses in any package
- C) Everywhere
- D) Only the same class

Q55. [Concept] Which is the correct syntax for a static import of `sqrt`?

- A) `import static java.lang.Math.sqrt;`
- B) `static import java.lang.Math.sqrt;`
- C) `import java.lang.static.Math;`
- D) `using static java.lang.Math;`

Q56. [Trap]

```
import java.util.*;
public class T {
  public static void main(String[] a) {
    ConcurrentHashMap<String,Integer> m = new ConcurrentHashMap<>();
  }
}
```

- A) Compiles; `*` imports sub-packages too
- B) Compile-time error: wildcard does not import sub-packages (java.util.concurrent)
- C) Runtime error
- D) Compiles because it is a Map

Q57. [Output]

```
class P { P() { System.out.print("P0 "); } P(int x) { System.out.print("P1
"); } }
class C extends P {
  C() { super(1); System.out.print("C0 "); }
  C(int x) { this(); System.out.print("C1 "); }
}
new C(5);
```

- A) P0 C0 C1
- B) P1 C0 C1
- C) P1 C1 C0
- D) P0 P1 C0 C1

Q58. [Scenario] (i) A Car and its Engine (the engine is destroyed when the car is scrapped). (ii) A Department and its Employees (employees continue to exist if the department is closed). Relationships are:

- A) (i) Composition, (ii) Aggregation
- B) (i) Aggregation, (ii) Composition
- C) (i) Inheritance, (ii) Aggregation
- D) (i) Association only, (ii) Inheritance

Q59. [Trap]

```
class A { private void p() { System.out.print("A"); } void call() { p(); } }
class B extends A { void p() { System.out.print("B"); } }
new B().call();
```

- A) A
- B) B
- C) Compile-time error
- D) Runtime error

Q60. [Output]

```
class A { String name = "A"; }
class B extends A {
  String name = "B";
  void p() { System.out.println(name + super.name + ((A) this).name); }
}
new B().p();
```

- A) BAA
- B) BBB
- C) ABB
- D) BAB

Q61. [Concept] Which statement about polymorphism is TRUE?

- A) Method overriding is resolved at compile time
- B) Method overloading is resolved at compile time using the declared types of arguments
- C) Field access is resolved at runtime based on the object
- D) Static methods are overridden at runtime

Q62. [Output] Hierarchy: Person → Employee → Manager.

```
System.out.print(new Manager() instanceof Employee);
System.out.print(" ");
System.out.print(new Employee() instanceof Manager);
```

- A) true true
- B) true false
- C) false true

- D) false false

Answer Key – Section 3

41. **B** – Methods are dynamically dispatched (B); fields are resolved by reference type (A.x = 10).
42. **A** – Static methods are hidden, not overridden; resolved by reference type.
43. **B** – Constructors run top-down: superclass first.
44. **B** – Superclass constructor runs before B's field initializers, so v is still 0 (overridden method called early).
45. **A** – Widening primitive conversion is tried before boxing.
46. **B** – The most specific type (String) is chosen. It would be ambiguous only if two unrelated types like String and StringBuilder were present.
47. **C** – An overriding method cannot throw broader checked exceptions. Wider access and covariant return types are fine.
48. **B** – B.f(B) is an overload, not an override. A reference of type A only sees f(A), which is not overridden.
49. **C** – Methods are checked against the reference type (Animal), which has no fetch().
50. **B** – The compiler allows the downcast (related types) but the object is not an Employee at runtime.
51. **A** – Pattern matching binds s; the length condition is true for "Java" (4 > 3).
52. **B** – instanceof is false for null; no exception.
53. **B** – Protected access outside the package works only through the subclass type (this/B), not through a plain A reference.
54. **A** – Package-private; not even subclasses in other packages.
55. **A** – The syntax is `import static ...`.
56. **B** – `java.util.*` covers only classes directly inside java.util.
57. **B** – C(int) → this() → C() → super(1) prints P1, then C0, then C1.
58. **A** – Composition = strong lifetime dependency; aggregation = weak.
59. **A** – Private methods are not inherited or overridden; A.call() calls A's own p().
60. **A** – name → B, super.name → A, ((A)this).name → A (field by reference type).
61. **B** – Overloading is compile-time; overriding is runtime; fields and static methods are not polymorphic.
62. **B** – A Manager is an Employee; an Employee is not a Manager.

Section 4: String Class, StringBuilder/Buffer, Wrapper Classes (Q63–Q82)

Q63. [Output]

```
String a = "hello", b = "hello", c = new String("hello");
System.out.print((a == b) + " " + (a == c) + " " + a.equals(c) + " " + (a ==
c.intern()));
```

- A) true false true true
- B) true true true true
- C) false false true true
- D) true false false true

Q64. [Trap]

```
String s1 = "ab";
String s2 = "a" + "b";
String s3 = "a";
String s4 = s3 + "b";
System.out.print((s1 == s2) + " " + (s1 == s4));
```

- A) true true
- B) true false
- C) false false
- D) false true

Q65. [Trap]

```
final String x = "a";
String z = "a";
System.out.print(((x + "b") == "ab") + " " + ((z + "b") == "ab"));
```

- A) true false
- B) false false
- C) true true
- D) false true

Q66. [Output]

```
String s = "java";
s.concat(" rocks");
s.toUpperCase();
System.out.println(s);
```

- A) java
- B) JAVA
- C) java rocks
- D) JAVA ROCKS

Q67. [Concept] How many String objects are created by `String s = new String("abc");` if "abc" is not yet in the pool?

- A) 1
- B) 2
- C) 3
- D) 0

Q68. [Output]

```
StringBuilder sb = new StringBuilder("ab");
StringBuilder sb2 = sb;
sb2.append("c").reverse();
System.out.println(sb);
```

- A) abc
- B) cba
- C) ab
- D) ba

Q69. [Trap]

```
System.out.println(new StringBuilder("a").equals(new StringBuilder("a")));
System.out.println(new StringBuilder("a").toString().equals(new
StringBuilder("a").toString()));
```

- A) true true
- B) false true
- C) false false
- D) true false

Q70. [Output]

```
String s = "Programming";
System.out.println(s.substring(3, 7) + s.indexOf('m') + s.charAt(s.length() - 1));
```

- A) gram6g
- B) ogram6g
- C) gram7g
- D) ram6g

Q71. [Trap]

```
String s = "a,b,,c,,";
System.out.println(s.split(",").length);
```

- A) 4
- B) 6
- C) 5
- D) 3

Q72. [Output]

```
String s = "  Hi  ";
System.out.println(s.trim().length() + s.length());
```

- A) 8
- B) 26
- C) 6
- D) 2

Q73. [Trap]

```
System.out.println("Apple".compareTo("apple"));
```

- A) -32
- B) 32
- C) -1
- D) 0

Q74. [Trap]

```
Integer a = 127, b = 127, c = 128, d = 128;
System.out.print((a == b) + " " + (c == d) + " " + c.equals(d));
```

- A) true true true
- B) true false true
- C) false false true
- D) true false false

Q75. [Scenario] A billing program does: `Map<String,Integer> stock = new HashMap<>(); int qty = stock.get("pen");` where "pen" was never added. What happens?

- A) qty = 0
- B) qty = null
- C) NullPointerException at runtime
- D) Compile-time error

Q76. [Output]

```
System.out.println(Integer.parseInt("12") + Integer.valueOf("3") + "4");
```

- A) 154
- B) 1234
- C) 19
- D) Compile-time error

Q77. [Trap]

```
Integer a = 5;
Long b = 5L;
System.out.print(a.equals(b) + " " + (a.longValue() == b));
```

- A) true true
- B) false true
- C) false false
- D) true false

Q78. [Trap] Which call throws NumberFormatException?

- A) `Integer.parseInt(" 12")`

- B) `Integer.parseInt("+12")`
- C) `Integer.parseInt("-0")`
- D) `Double.parseDouble(" 3.5 ")`

Q79. [Output]

```
StringBuilder sb = new StringBuilder("Java");
sb.insert(2, "--").delete(0, 1).replace(0, 1, "X").append(1);
System.out.println(sb);
```

- A) X--va1
- B) X-va1
- C) a--va1
- D) XJ--va1

Q80. [Scenario] Several threads append log lines to one shared text buffer. Which class is the safest choice?

- A) String
- B) StringBuilder
- C) StringBuffer
- D) Character[]

Q81. [Trap]

```
String s = null;
s += "a";
System.out.println(s);
```

- A) a
- B) nulla
- C) NullPointerException
- D) null

Q82. [Concept] For `"Aa"` and `"BB"`, which is true?

- A) hashCode() values are the same but equals() is false
- B) hashCode() values differ
- C) equals() is true
- D) == is true

Answer Key – Section 4

63. **A** – Literals share a pool entry; new creates a separate heap object; intern() returns the pooled one.
64. **B** – "a"+"b" is a compile-time constant (pooled); s3 + "b" is computed at runtime (new object).
65. **A** – A final String initialized with a constant makes the concatenation a constant expression.
66. **A** – Strings are immutable; the results of concat/toUpperCase were discarded.
67. **B** – One in the pool (literal) and one on the heap (new).
68. **B** – Both refer to the same builder: "abc" reversed is "cba".
69. **B** – StringBuilder does not override equals(); it uses identity.
70. **A** – substring(3,7)="gram", indexOf('m')=6, last char 'g'.
71. **A** – Trailing empty strings are removed: [a, b, "", c].
72. **A** – 2 + 6 = 8 (integer addition, not concatenation).
73. **A** – 'A'(65) - 'a'(97) = -32.
74. **B** – Integer cache covers -128 to 127; 128 creates different objects.
75. **C** – get() returns null; auto-unboxing null throws NPE.
76. **A** – 12 + 3 = 15, then + "4" gives "154".
77. **B** – Integer.equals(Long) is false (different types); the primitive comparison is true.
78. **A** – Integer.parseInt does not accept spaces; "+12" and "-0" are valid; Double.parseDouble trims whitespace.
79. **A** – "Ja--va" → "a--va" → "X--va" → "X--va1".
80. **C** – StringBuffer's methods are synchronized.
81. **B** – null + "a" gives "nulla" (String.valueOf(null reference)).
82. **A** – Both hash to 2112: a classic collision.

Section 5: Enum, Sealed, Final, Pattern Matching, Inner Classes (Q83–Q94)

Q83. [Output]

```
enum Level {
  LOW(1), HIGH(3);
  private final int v;
  Level(int v) { this.v = v; }
  int v() { return v; }
}
System.out.println(Level.HIGH.v() + Level.HIGH.ordinal());
```

- A) 3
- B) 4
- C) 31
- D) Compile-time error

Q84. [Scenario] A form sends the department as text "hr" and the code calls `Dept.valueOf("hr")` where the enum constant is `HR`. What happens?

- A) Returns null
- B) Returns Dept.HR
- C) Throws IllegalArgumentException
- D) Throws NullPointerException

Q85. [Concept] Which statement about enums is FALSE?

- A) An enum can implement an interface
- B) An enum constructor can be declared public
- C) Enum constants can have constant-specific class bodies
- D) Every enum implicitly extends java.lang.Enum

Q86. [Trap]

```
enum Dept { HR, IT, FIN }
String r = switch (d) {
  case HR -> "h";
  case IT -> "i";
  case FIN -> "f";
};
```

- A) Compiles: a switch expression over an enum is exhaustive when all constants are covered
- B) Compile-time error: default is mandatory

- C) Compile-time error: break is mandatory
- D) Runtime error

Q87. [Trap]

```
final int[] arr = {1, 2};
arr[0] = 9;        // line 1
arr = new int[3];  // line 2
```

- A) Only line 1 fails
- B) Only line 2 fails
- C) Both fail
- D) Neither fails

Q88. [Concept] Which declaration causes a compile-time error?

- A) A final class that contains a static method
- B) `abstract final class X {}`
- C) A final method inside an abstract class
- D) A static final variable

Q89. [Output]

```
sealed interface Shape permits Circle, Square {}
record Circle(double r) implements Shape {}
record Square(double s) implements Shape {}
final class Triangle implements Shape {}
```

- A) Compiles
- B) Compile-time error: Triangle is not listed in permits
- C) Runtime error
- D) Warning only

Q90. [Trap] `sealed class Shape permits Sq {}` and then `class Sq extends Shape {}`. What is the result?

- A) Compiles; Sq is implicitly final
- B) Compile-time error: Sq must be final, sealed or non-sealed
- C) Compiles; Sq is implicitly non-sealed
- D) Runtime error

Q91. [Output]

```
static void p(Object o) {
  if (!(o instanceof Integer i)) { return; }
  System.out.println(i + 1);
}
// main
p(5);
```

- A) 6
- B) Compile-time error: i is not in scope
- C) 51
- D) ClassCastException

Q92. [Output]

```
class Outer {
  int x = 10;
  class Inner {
    int x = 20;
    void p() { int x = 30; System.out.println(x + " " + this.x + " " +
Outer.this.x); }
  }
}
new Outer().new Inner().p();
```

- A) 30 20 10
- B) 10 20 30
- C) 30 30 30
- D) Compile-time error

Q93. [Concept] Which correctly creates an object of a regular (non-static) inner class `Inner` of `Outer` from `main`?

- A) `Outer.Inner i = new Outer.Inner();`
- B) `Inner i = new Inner();`
- C) `Outer.Inner i = new Outer().new Inner();`
- D) `Inner i = Inner.new Outer();`

Q94. [Scenario]

```
int count = 0;
Runnable r = new Runnable() {
  public void run() { count++; }
};
```

- A) Works; count becomes 1 after run()
- B) Compile-time error: variable captured by an anonymous class must be effectively final
- C) Runtime exception
- D) Prints 0

Answer Key – Section 5

83. **B** – 3 + ordinal 1 = 4.
84. **C** – valueOf is case-sensitive and throws IllegalArgumentException when no constant matches.
85. **B** – Enum constructors are implicitly private; public is not allowed.
86. **A** – Covering every constant makes a switch expression exhaustive without default.
87. **B** – final fixes the reference, not the array's contents.
88. **B** – abstract and final contradict each other.
89. **B** – Only classes listed in permits may implement a sealed type.
90. **B** – A permitted subclass must be final, sealed or non-sealed (records and enums are implicitly final).
91. **A** – Flow scoping: i is in scope after the early return.
92. **A** – Local x = 30, this.x = 20, Outer.this.x = 10.
93. **C** – An inner class needs an enclosing instance.
94. **B** – Captured locals must be final or effectively final.

Section 6: Abstract Class, Interfaces, Garbage Collection (Q95–Q110)

Q95. [Output]

```
abstract class Device {
  Device() { System.out.print("D "); }
  abstract void on();
}
class Phone extends Device {
  Phone() { System.out.print("P "); }
  void on() { }
}
new Phone();
```

- A) P D
- B) D P
- C) P
- D) D

Q96. [Concept] Which declaration is valid?

- A) An abstract method with a body
- B) `private abstract void f();`
- C) An abstract class with no abstract methods
- D) `abstract static void f();`

Q97. [Trap]

```
interface I { int X = 5; }
class C implements I { void f() { X = 6; } }
```

- A) Compiles; X becomes 6
- B) Compile-time error: cannot assign to a final variable
- C) Runtime error
- D) Prints 5

Q98. [Trap]

```
interface I { default void hi() { System.out.print("I"); } }
interface J { default void hi() { System.out.print("J"); } }
class C implements I, J { }
```

- A) Compiles; prints I when called
- B) Compiles; prints J when called

- C) Compile-time error: C inherits unrelated defaults for hi()
- D) Runtime ambiguity

Q99. [Output]

```
interface I { default void hi() { System.out.print("I"); } }
class P { public void hi() { System.out.print("P"); } }
class C extends P implements I { }
new C().hi();
```

- A) I
- B) P
- C) IP
- D) Compile-time error

Q100. [Trap] `interface I { static void util() { } }` and `class C implements I { }`. Which call compiles?

- A) `new C().util();`
- B) `C.util();`
- C) `I.util();`
- D) None of them

Q101. [Concept] Which can be declared inside an interface in Java 17?

- A) A private method with a body
- B) A protected method
- C) An instance field
- D) A constructor

Q102. [Concept] Which interface qualifies as a functional interface?

- A) Two abstract methods
- B) One abstract method, one default method, and a re-declaration of `boolean equals(Object o)`
- C) No abstract methods
- D) Two unrelated abstract methods and one static method

Q103. [Scenario] In a Smart Device system, a Phone and an ElectricCar (unrelated hierarchies) can both be charged. How should `Rechargeable` be modeled?

- A) Abstract class, since all devices charge
- B) Interface, since it is a capability shared by unrelated classes
- C) Final class
- D) Enum

Q104. [Concept] Which compiles?

- A) `class X extends A, B` (A and B are classes)
- B) `interface Z extends I, J`
- C) `class Y implements A` (A is a class)
- D) `interface W implements I`

Q105. [Trap]

```
interface I { void a(); void b(); }
abstract class P implements I { public void a() { } }
class C extends P { public void a() { } }
```

- A) Compiles
- B) Compile-time error: C must implement b()
- C) Compile-time error: P must implement b()
- D) Compile-time error: a() cannot be redefined

Q106. [Scenario] How many objects are eligible for GC after the last line?

```
Object a = new Object();   // O1
Object b = new Object();   // O2
Object c = a;
a = null;
b = c;
c = null;
```

- A) 0
- B) 1
- C) 2
- D) 3

Q107. [Trap]

```
class Node { Node next; }
Node n1 = new Node(), n2 = new Node();
n1.next = n2;
n2.next = n1;
n1 = null;
n2 = null;
```

- A) Neither is eligible because they reference each other
- B) Both are eligible (island of isolation)
- C) Only n1's object is eligible
- D) Only n2's object is eligible

Q108. [Concept] `System.gc()` does what?

- A) Guarantees immediate garbage collection
- B) Suggests (requests) that the JVM run garbage collection
- C) Forces finalize() on all objects
- D) Clears the stack

Q109. [Concept] Which statement about `finalize()` is correct?

- A) It is guaranteed to run before the program exits
- B) The JVM calls it at most once per object
- C) It is called every time GC runs
- D) It is called on local variables when a method ends

Q110. [Scenario] Which situation can cause a memory leak in Java despite garbage collection?

- A) Local variables inside a method
- B) A static collection that keeps references to objects that are never removed
- C) Setting a reference to null
- D) Calling System.gc()

Answer Key – Section 6

95. **B** – An abstract class has a constructor; it runs first via super().
96. **C** – An abstract class may have zero abstract methods. Abstract methods cannot have a body, be private or static.
97. **B** – Interface fields are public static final.

98. **C** – You must override hi() in C (and may call I.super.hi()).
99. **B** – The class wins rule: a concrete class method beats an interface default.
100. **C** – Static interface methods are called only through the interface name.
101. **A** – Private methods are allowed since Java 9. Members are public (or private), fields are constants, no constructors.
102. **B** – Methods matching public Object methods don't count as abstract; default and static methods don't count either.
103. **B** – Capabilities across unrelated classes are modeled with interfaces; shared state and partial implementation suit an abstract class.
104. **B** – Interfaces can extend multiple interfaces.
105. **B** – P is abstract so it can skip b(), but the concrete class C must implement it.
106. **B** – O1 is still reachable via b; only O2 is eligible.
107. **B** – Unreachable from any live root, so both are eligible.
108. **B** – It is only a request.
109. **B** – At most once (and finalize() is deprecated).
110. **B** – Reachable but unused objects are never collected.

Section 7: Exception Handling (Q111–Q126)

Q111. [Output]

```
static int f() {
  try { return 1; } finally { System.out.print("F "); }
}
System.out.println(f());
```

- A) 1 F
- B) F 1
- C) 1
- D) F

Q112. [Trap]

```
static int g() {
  int x = 1;
  try { return x; } finally { x = 2; }
}
System.out.println(g());
```

- A) 1
- B) 2
- C) 3
- D) Compile-time error

Q113. [Trap]

```
try { int[] a = new int[2]; a[2] = 1; }
catch (Exception e) { System.out.print("E "); }
catch (ArrayIndexOutOfBoundsException e) { System.out.print("A "); }
```

- A) E
- B) A
- C) Compile-time error: exception already caught
- D) Runtime error

Q114. [Trap] What is wrong with `catch (IOException | FileNotFoundException e)`?

- A) Nothing, it compiles
- B) Compile-time error: alternatives in a multi-catch cannot be related by subclassing
- C) Runtime error
- D) e becomes non-final

Q115. [Output]

```
static void a() { try { b(); } finally { System.out.print("A "); } }
static void b() { throw new RuntimeException("x"); }
// main
try { a(); } catch (Exception e) { System.out.print("M "); }
```

- A) M A
- B) A M
- C) M
- D) A

Q116. [Trap] Which of these is a CHECKED exception?

- A) IllegalStateException
- B) ArrayStoreException
- C) InterruptedException
- D) NumberFormatException

Q117. [Scenario] Banking code:

```
class InsufficientBalanceException extends Exception { }
void withdraw(double amt) {
  if (amt > bal) throw new InsufficientBalanceException();
}
```

- A) Compiles and works
- B) Compile-time error: unreported checked exception
- C) Runtime error
- D) Only a warning

Q118. [Output]

```
class R implements AutoCloseable {
  String n;
  R(String n) { this.n = n; System.out.print("open" + n + " "); }
  public void close() { System.out.print("close" + n + " "); }
}
try (R a = new R("1"); R b = new R("2")) { System.out.print("body "); }
```

- A) open1 open2 body close1 close2
- B) open1 open2 body close2 close1
- C) open1 body open2 close2 close1
- D) open1 open2 close2 close1 body

Q119. [Trap]

```
try { throw new RuntimeException("try"); }
finally { throw new IllegalStateException("fin"); }
```

Which exception reaches the caller?

- A) RuntimeException("try")
- B) IllegalStateException("fin")

- C) Both, chained
- D) None

Q120. [Output]

```
try { System.out.print("1 "); int x = 5 / 0; System.out.print("2 "); }
catch (ArithmeticException e) { System.out.print("3 "); }
finally { System.out.print("4 "); }
System.out.print("5");
```

- A) 1 2 4 5
- B) 1 3 4 5
- C) 1 3 5
- D) 1 4 5

Q121. [Trap]

```
try { System.exit(0); }
finally { System.out.print("F"); }
```

- A) F
- B) Prints nothing
- C) Throws an exception
- D) Compile-time error

Q122. [Scenario] A method recurses infinitely inside `try { ... } catch (Exception e) { log(e); }`. What happens?

- A) The exception is caught and logged
- B) The program terminates with an uncaught StackOverflowError
- C) Compile-time error
- D) It returns normally

Q123. [Trap]

```
throw new RuntimeException();
System.out.println("x");
```

- A) Prints x then throws
- B) Throws only
- C) Compile-time error: unreachable statement

- D) Warning only

Q124. [Trap]

```
interface Rd { void read() throws IOException; }
class R implements Rd { public void read() { } }
R r = new R();   r.read();   // call 1, no try
Rd q = new R();  q.read();   // call 2, no try
```

- A) Only call 1 fails
- B) Only call 2 fails
- C) Both fail
- D) Neither fails

Q125. [Concept] Which compiles?

- A) `try { }`
- B) `try { } finally { }`
- C) `try { } catch`
- D) `catch (Exception e) { }` alone

Q126. [Scenario] A banking app parses an account number with `Integer.parseInt("abc")`. Which is true?

- A) NumberFormatException extends IllegalArgumentException, so catch (IllegalArgumentException e) handles it
- B) NumberFormatException extends IOException
- C) NumberFormatException is a checked exception
- D) NumberFormatException extends Error

Answer Key – Section 7

111. **B** – finally runs (prints F) before the method returns; then println prints 1.
112. **A** – The return value is saved before finally runs; changing x later does not affect it.
113. **C** – A subclass catch after its superclass catch is unreachable.
114. **B** – Multi-catch alternatives cannot be subclass-related.
115. **B** – finally in a() runs while the exception propagates, then main catches.
116. **C** – InterruptedException is checked. The others are RuntimeExceptions.
117. **B** – Must handle or declare with throws.

118. **B** – Resources close in reverse order after the body.
119. **B** – The exception thrown in finally replaces the earlier one.
120. **B** – 1 → exception → catch prints 3 → finally 4 → 5.
121. **B** – System.exit halts the JVM; finally does not run.
122. **B** – StackOverflowError is an Error, not an Exception.
123. **C** – Code after an unconditional throw is unreachable.
124. **B** – The reference type decides: Rd declares IOException, R does not.
125. **B** – try needs a catch, finally or resources.
126. **A** – NumberFormatException is an unchecked subclass of IllegalArgumentException.

Section 8: File I/O, Serialization, Object Class, Date-Time API (Q127–Q136)

Q127. [Output]

```
class U implements Serializable {
  String name = "Ann";
  transient String pwd = "123";
}
// serialize a U object to a file, then deserialize it into u2
System.out.println(u2.name + " " + u2.pwd);
```

- A) Ann null
- B) Ann 123
- C) null null
- D) NotSerializableException

Q128. [Scenario] `Employee implements Serializable` has a field `Address addr`, but `Address` does not implement Serializable. What happens on `writeObject(emp)`?

- A) Works; addr is skipped
- B) NotSerializableException at runtime
- C) Compile-time error
- D) addr silently becomes null

Q129. [Trap]

```
class P { P() { System.out.print("P "); } }
class C extends P implements Serializable { C() { System.out.print("C "); } }
```

What is printed while DESERIALIZING a C object (not while creating it)?

- A) P C
- B) C
- C) P
- D) Nothing

Q130. [Scenario] A logging program uses `new FileWriter("log.txt")` and every run erases the old log lines. Why?

- A) The writer was opened without the append flag, so it truncates the file
- B) flush() was not called
- C) The file is locked
- D) Wrong charset

Q131. [Concept] Best class for copying a binary file such as a JPEG?

- A) FileReader
- B) FileInputStream (byte stream)
- C) BufferedReader
- D) Scanner

Q132. [Scenario] A program writes 100 lines using a `BufferedWriter` and then ends without calling `flush()` or `close()`. What is the likely result?

- A) All data is always on disk
- B) Part or all of the buffered data may never be written
- C) Compile-time error
- D) NullPointerException

Q133. [Trap]

```
class P {
  int id; P(int id) { this.id = id; }
  public boolean equals(P o) { return id == o.id; }
}
List<P> l = List.of(new P(1));
System.out.println(l.contains(new P(1)));
```

- A) true
- B) false
- C) Compile-time error
- D) NullPointerException

Q134. [Concept] Which statement about the equals()/hashCode() contract is correct?

- A) If a.equals(b) is true, then a.hashCode() must equal b.hashCode()
- B) If two hash codes are equal, the objects must be equal
- C) hashCode() must return a unique value per object
- D) hashCode() must return a random value

Q135. [Output]

```
LocalDate d = LocalDate.of(2024, 1, 31);
System.out.println(d.plusMonths(1));
```

- A) 2024-02-31
- B) 2024-02-29
- C) 2024-03-02
- D) DateTimeException

Q136. [Trap]

```
LocalDate d = LocalDate.parse("2026-1-5");
```

- A) 2026-01-05
- B) DateTimeParseException
- C) null
- D) 2026-1-5

Answer Key – Section 8

127. **A** – transient fields are skipped and get default values.
128. **B** – Every non-transient object field must be serializable.
129. **C** – On deserialization, constructors of serializable classes are skipped; the first non-serializable superclass's no-arg constructor runs.
130. **A** – Use `new FileWriter(name, true)` to append.
131. **B** – Byte streams for binary data; Readers/Writers are for text.

132. **B** – Buffered data must be flushed or closed.
133. **B** – equals(P) overloads rather than overrides equals(Object); contains() uses equals(Object).
134. **A** – Equal objects need equal hash codes; the reverse is not required.
135. **B** – The day is clamped to the last valid day of February (leap year).
136. **B** – ISO format needs zero-padded month and day.

Section 9: Generics, Collections, Hashing, Sorting (Q137–Q164)

Q137. [Trap]

```
List<Object> lo = new ArrayList<String>();
```

- A) Compiles
- B) Compile-time error: generics are invariant
- C) Runtime ClassCastException
- D) Compiles with a warning

Q138. [Trap]

```
List<? extends Number> l = new ArrayList<Integer>();
l.add(5);
```

- A) Compiles and adds 5
- B) Compile-time error
- C) Runtime error
- D) Adds null

Q139. [Trap]

```
List<? super Integer> l = new ArrayList<Number>();
l.add(5);               // line 1
Integer x = l.get(0);   // line 2
```

- A) Line 1 fails
- B) Line 2 fails
- C) Both fail
- D) Neither fails

Q140. [Output]

```
static <T extends Comparable<T>> T max(T a, T b) { return a.compareTo(b) > 0
? a : b; }
System.out.println(max("apple", "banana") + max(3, 7));
```

- A) banana7
- B) apple3
- C) Compile-time error
- D) banana10

Q141. [Trap]

```
List<String> a = new ArrayList<>();
List<Integer> b = new ArrayList<>();
System.out.println(a.getClass() == b.getClass());
```

- A) true
- B) false
- C) Compile-time error
- D) ClassCastException

Q142. [Trap] In one class: `void f(List<String> l)` and `void f(List<Integer> l)`. Result?

- A) Valid overloading
- B) Compile-time error: same erasure
- C) Runtime error
- D) Only a warning

Q143. [Concept] In `class Box<T> { ... }`, which line is illegal?

- A) `private T value;`
- B) `public T get() { return value; }`
- C) `static T shared;`
- D) `<U> void m(U u) { }`

Q144. [Output]

```
List<Integer> l = new ArrayList<>(List.of(10, 20, 30, 1));
l.remove(1);
l.remove(Integer.valueOf(1));
System.out.println(l);
```

- A) [10, 30]
- B) [10, 20, 30]
- C) [20, 30]
- D) [10, 30, 1]

Q145. [Trap]

```
List<Integer> l = new ArrayList<>(List.of(1, 2, 3, 4));
for (Integer x : l) { if (x == 3) l.remove(x); }
System.out.println(l);
```

- A) ConcurrentModificationException
- B) [1, 2, 4] with no exception
- C) [1, 2, 3]
- D) IndexOutOfBoundsException

Q146. [Trap]

```
List<Integer> l = Arrays.asList(1, 2, 3);
l.set(0, 9);
l.add(4);
```

- A) Both work
- B) set works; add throws UnsupportedOperationException
- C) set throws; add works
- D) Both throw

Q147. [Scenario] A Task Manager adds tasks at the end and mostly reads a task by index (get(i)) in loops. Best List?

- A) ArrayList
- B) LinkedList
- C) Both are identical
- D) Vector only

Q148. [Concept] Which statement is TRUE?

- A) ArrayList methods are synchronized
- B) Vector methods are synchronized
- C) HashMap is synchronized
- D) LinkedList is synchronized

Q149. [Output]

```
class E {
  int id; String n;
  E(int id, String n) { this.id = id; this.n = n; }
  public boolean equals(Object o) { return o instanceof E e && e.id == id; }
  public int hashCode() { return Integer.hashCode(id); }
}
Set<E> s = new HashSet<>();
s.add(new E(1, "A")); s.add(new E(1, "B")); s.add(new E(2, "A"));
System.out.println(s.size());
```

- A) 1
- B) 2
- C) 3
- D) 4

Q150. [Trap] Same class E as Q149 but only equals() is overridden (no hashCode()). What is `s.size()` after adding two `new E(1, "A")` objects to a HashSet?

- A) Always 1
- B) Typically 2
- C) Exception
- D) Compile-time error

Q151. [Trap] E from Q149 (with both methods based on id):

```
Map<E, String> m = new HashMap<>();
E k = new E(1, "A");
m.put(k, "v");
k.id = 2;
System.out.println(m.get(k));
```

- A) v

- B) null
- C) Exception
- D) Compile-time error

Q152. [Output]

```
Map<String, Integer> m = new HashMap<>();
System.out.print(m.put("a", 1) + " ");
System.out.print(m.put("a", 2) + " ");
System.out.print(m.put(null, 3) + " ");
System.out.print(m.get(null));
```

- A) null 1 null 3
- B) 1 2 3 3
- C) null null null 3
- D) NullPointerException

Q153. [Trap]

```
Map<String, Integer> m = new TreeMap<>();
m.put("b", 1);
m.put("a", 2);
m.put(null, 3);
```

- A) Works; null sorts first
- B) NullPointerException
- C) ClassCastException
- D) Compile-time error

Q154. [Output]

```
Set<String> s = new TreeSet<>(List.of("pear", "apple", "fig"));
System.out.println(s);
```

- A) [pear, apple, fig]
- B) [apple, fig, pear]
- C) [fig, apple, pear]
- D) Order is unpredictable

Q155. [Scenario] `TreeSet<Student> ts = new TreeSet<>(); ts.add(new Student(1));` where Student does not implement Comparable and no Comparator is given.

- A) Compile-time error
- B) ClassCastException at runtime on add
- C) Works; insertion order is kept
- D) NullPointerException

Q156. [Trap]

```
Set<String> s = new TreeSet<>(String.CASE_INSENSITIVE_ORDER);
s.add("Java"); s.add("JAVA"); s.add("java");
System.out.println(s.size() + " " + s);
```

- A) 3 [JAVA, Java, java]
- B) 1 [Java]
- C) 1 [java]
- D) 3 [Java, JAVA, java]

Q157. [Concept] A default `new HashMap<>()` (capacity 16, load factor 0.75). On which put() does the first resize happen?

- A) 12th
- B) 13th
- C) 16th
- D) 17th

Q158. [Concept] In Java 8+, a HashMap bucket's linked list is converted to a red-black tree when:

- A) It has at least 8 nodes and the table capacity is at least 64
- B) It has 4 nodes
- C) It has 16 nodes regardless of capacity
- D) Any collision occurs

Q159. [Output]

```
Set<String> s = new HashSet<>();
s.add("Aa"); s.add("BB");
System.out.println(s.size());
```

- A) 1
- B) 2
- C) Exception
- D) 0

Q160. [Concept] Which statement is TRUE?

- A) HashMap and Hashtable both allow null keys
- B) Hashtable does not allow null keys or values
- C) TreeMap allows many null keys
- D) LinkedHashMap does not allow null values

Q161. [Trap]

```
List<String> base = new ArrayList<>(List.of("a"));
List<String> ro = Collections.unmodifiableList(base);
base.add("b");
System.out.println(ro);
```

- A) [a]
- B) [a, b]
- C) UnsupportedOperationException
- D) []

Q162. [Output] (Pseudo-code)

```
record S(String name, String dept, int marks) { }
List<S> list = new ArrayList<>(List.of(
  new S("Ravi", "IT", 85), new S("Anu", "CS", 90),
  new S("Sam", "CS", 95), new S("Tej", "IT", 90)));
list.sort(Comparator.comparing(S::dept)
    .thenComparing(S::marks, Comparator.reverseOrder()));
// print names in order
```

- A) Sam Anu Tej Ravi
- B) Anu Sam Ravi Tej
- C) Sam Anu Ravi Tej
- D) Anu Sam Tej Ravi

Q163. [Trap]

```
Comparator<Integer> c = (a, b) -> a - b;
System.out.println(c.compare(Integer.MIN_VALUE, 1));
```

- A) A negative number
- B) 2147483647 (a wrong positive result due to overflow)
- C) 0
- D) ArithmeticException

Q164. [Output]

```
Queue<Integer> pq = new PriorityQueue<>(List.of(5, 1, 4, 2));
System.out.println(pq.poll() + " " + pq.peek());
```

- A) 5 1
- B) 1 2
- C) 1 4
- D) 2 1

Answer Key – Section 9

137. **B** – List<String> is not a List<Object>.
138. **B** – With `? extends`, the list is read-only (only null can be added).
139. **B** – `? super Integer` allows add(Integer), but get() returns Object.
140. **A** – "banana" + 7 = "banana7".
141. **A** – Type erasure: both are ArrayList at runtime.
142. **B** – Both erase to f(List).
143. **C** – Static members cannot use the class type parameter.
144. **A** – remove(1) removes index 1 (20); remove(Integer) removes the value 1.
145. **B** – Removing the second-last element ends the loop before the fail-fast check (a classic trap). Removing most other elements throws ConcurrentModificationException.
146. **B** – Arrays.asList is a fixed-size list backed by the array.
147. **A** – ArrayList gives O(1) get(i); LinkedList is O(n).
148. **B** – Vector is a legacy synchronized list.
149. **B** – id 1 duplicates; set = {E1, E2}.

150. **B** – Without hashCode() the identity hashes usually differ, so both are stored (a contract violation).
151. **B** – The hash changed after insertion; lookup goes to a different bucket.
152. **A** – put returns the previous value; HashMap allows one null key.
153. **B** – TreeMap with natural ordering cannot compare null.
154. **B** – TreeSet sorts naturally.
155. **B** – TreeSet casts to Comparable when adding.
156. **B** – TreeSet uses the comparator for uniqueness; the first inserted element stays.
157. **B** – Threshold is 12; the resize happens when the size exceeds it (13th entry).
158. **A** – Treeification needs chain length of 8 or more and capacity of 64 or more; otherwise the table resizes.
159. **B** – "Aa" and "BB" collide in hashCode but are unequal, so both are stored.
160. **B** – Hashtable rejects nulls; HashMap allows one null key; TreeMap (natural) allows none.
161. **B** – unmodifiableList is a read-only view of the original list, not a copy.
162. **A** – CS: Sam 95, Anu 90; then IT: Tej 90, Ravi 85.
163. **B** – MIN_VALUE - 1 overflows to MAX_VALUE. Use Integer.compare(a, b).
164. **B** – The min-heap polls 1; the next head is 2.

Section 10: Lambda, Functional Interfaces, Stream API (Q165–Q182)

Q165. [Concept] Which lambda is INVALID for `Function<Integer,Integer>`?

- A) `x -> x * 2`
- B) `(x) -> { return x * 2; }`
- C) `x -> { x * 2; }`
- D) `(Integer x) -> x * 2`

Q166. [Trap]

```
int base = 10;
Function<Integer, Integer> f = x -> x + base;
base = 20;
```

- A) Works; f.apply(1) gives 21

- B) Compile-time error: base must be effectively final
- C) f.apply(1) gives 11
- D) Runtime error

Q167. [Output]

```
Predicate<String> p1 = s -> s.length() > 3;
Predicate<String> p2 = s -> s.startsWith("a");
System.out.print(p1.and(p2).test("apple") + " " +
p1.negate().or(p2).test("beet"));
```

- A) true true
- B) true false
- C) false true
- D) false false

Q168. [Output]

```
Function<Integer, Integer> d = x -> x * 2;
Function<Integer, Integer> a = x -> x + 3;
System.out.print(d.andThen(a).apply(5) + " " + d.compose(a).apply(5));
```

- A) 13 16
- B) 16 13
- C) 13 13
- D) 16 16

Q169. [Scenario] `() -> LocalDate.now()` fits which built-in functional interface?

- A) Consumer<LocalDate>
- B) Supplier<LocalDate>
- C) Function<LocalDate,LocalDate>
- D) Predicate<LocalDate>

Q170. [Concept] `System.out::println` used as a Consumer is which kind of method reference?

- A) Static method reference
- B) Instance method reference of a particular (bound) object
- C) Instance method reference of an arbitrary object of a type

- D) Constructor reference

Q171. [Output]

```
Optional<Integer> r = Stream.of(1, 2, 3, 4)
  .filter(x -> { System.out.print("f" + x + " "); return x % 2 == 0; })
  .map(x -> { System.out.print("m" + x + " "); return x * x; })
  .findFirst();
```

- A) f1 f2 f3 f4 m2 m4
- B) f1 f2 m2
- C) m2 f1 f2
- D) Nothing

Q172. [Trap]

```
Stream.of(1, 2, 3, 4)
  .filter(x -> { System.out.print("f" + x + " "); return x % 2 == 0; })
  .map(x -> x * x);
```

- A) f1 f2 f3 f4
- B) Nothing is printed (no terminal operation)
- C) f1 f2
- D) Compile-time error

Q173. [Trap]

```
Stream<Integer> s = List.of(1, 2, 3).stream();
s.forEach(System.out::print);
s.forEach(System.out::print);
```

- A) 123123
- B) 123 and then IllegalStateException
- C) Compile-time error
- D) 123 then NullPointerException

Q174. [Output]

```
List<String> r = Stream.of("a", "bb", "cc", "ddd")
  .filter(s -> s.length() > 1)
  .map(String::toUpperCase)
  .collect(Collectors.toList());
System.out.println(r);
```

- A) [BB, CC, DDD]
- B) [A, BB, CC, DDD]
- C) [bb, cc, ddd]
- D) [BB, CC]

Q175. [Output]

```
List<List<Integer>> ll = List.of(List.of(1, 2), List.of(3), List.of());
System.out.println(ll.stream().flatMap(List::stream).map(x -> x *
2).reduce(0, Integer::sum));
```

- A) 6
- B) 12
- C) 3
- D) Compile-time error

Q176. [Trap]

```
System.out.println(Stream.of(1, 2, 3).reduce(Integer::sum));
```

- A) 6
- B) Optional[6]
- C) Optional.6
- D) Compile-time error

Q177. [Scenario] Employee names grouped by name length:

```
Map<Integer, List<String>> m = Stream.of("a", "bb", "cc", "d")
  .collect(Collectors.groupingBy(String::length));
System.out.println(m);
```

- A) {1=[a, d], 2=[bb, cc]}
- B) {2=[bb, cc], 1=[a, d]}
- C) {1=[a], 2=[bb, cc]}

- D) [[a, d], [bb, cc]]

Q178. [Output]

```
System.out.println(Stream.of(1, 2, 3, 4, 5)
  .collect(Collectors.partitioningBy(x -> x % 2 == 0,
Collectors.counting())));
```

- A) {false=3, true=2}
- B) {true=2, false=3}
- C) {2=true, 3=false}
- D) {even=2, odd=3}

Q179. [Output]

```
Stream.of(5, 3, 5, 1,
3).distinct().sorted().skip(1).limit(1).forEach(System.out::println);
```

- A) 1
- B) 3
- C) 5
- D) 3 5

Q180. [Trap] (Java 17)

```
System.out.println(Stream.of(1, 2, 3).peek(x -> System.out.print(x + "
")).count());
```

- A) 1 2 3 3
- B) 3 (peek is not executed because the count is known from the source)
- C) 3 1 2 3
- D) Compile-time error

Q181. [Concept] Which statement is TRUE?

- A) `Optional.of(null)` throws NullPointerException
- B) `Optional.ofNullable(null).orElse("x")` throws an exception
- C) `Optional.empty().get()` returns null
- D) `Optional.of("a").isEmpty()` is true

Q182. [Concept] Which statement is TRUE?

- A) `parallelStream().forEach()` preserves encounter order
- B) `forEachOrdered()` preserves encounter order even on a parallel stream
- C) Parallel streams are always faster
- D) `stream()` runs on multiple threads by default

Answer Key – Section 10

165. **C** – A block lambda needs a return statement; `x * 2;` is not a valid statement.
166. **B** – Captured locals must be effectively final; reassigning breaks that.
167. **B** – "apple": true and true gives true. "beet": negate(len>3) is false, startsWith(a) is false, so false.
168. **A** – andThen: d then a = 10+3 = 13; compose: a then d = (5+3)*2 = 16.
169. **B** – No arguments and returns a value: Supplier.
170. **B** – It is bound to the System.out object.
171. **B** – Streams process element by element and findFirst short-circuits.
172. **B** – Intermediate operations are lazy.
173. **B** – A stream can be consumed only once.
174. **A** – Filter keeps length > 1; map uppercases.
175. **B** – (1+2+3) x 2 = 12.
176. **B** – reduce without identity returns an Optional.
177. **A** – groupingBy gives a HashMap; keys 1 and 2 print in that order.
178. **A** – partitioningBy gives false first, then true; counts are 3 and 2.
179. **B** – [5,3,1] → sorted [1,3,5] → skip 1 → [3,5] → limit 1 → 3.
180. **B** – Since Java 9, count() may skip the pipeline for SIZED sources, so peek() is skipped.
181. **A** – Optional.of rejects null; get() on empty throws NoSuchElementException.
182. **B** – forEach on a parallel stream has no order guarantee.

Section 11: Multithreading, Synchronization, Reflection (Q183–Q192)

Q183. [Trap]

```
Thread t = new Thread(() ->
System.out.print(Thread.currentThread().getName()));
t.run();
```

- A) Thread-0
- B) main
- C) Nothing
- D) Exception

Q184. [Trap]

```
Thread t = new Thread(() -> { });
t.start();
t.start();
```

- A) Runs twice
- B) IllegalThreadStateException
- C) Second call is ignored silently
- D) Compile-time error

Q185. [Output]

```
Thread t = new Thread(() -> System.out.print("T "));
t.start();
t.join();
System.out.print("M ");
```

- A) T M
- B) M T
- C) Either order
- D) M

Q186. [Scenario] Two threads each execute `count++` 1000 times on a shared non-volatile int (no synchronization). The final value is:

- A) Always 2000
- B) At most 2000, and often less (race condition)
- C) Always 1000
- D) An exception is thrown

Q187. [Trap] Two threads call a synchronized INSTANCE method, but on two DIFFERENT objects. What happens?

- A) One blocks the other
- B) They run concurrently (the lock is per object)

- C) Deadlock
- D) Compile-time error

Q188. [Trap] Calling `obj.wait()` outside a synchronized block on obj throws:

- A) InterruptedException
- B) IllegalMonitorStateException
- C) NullPointerException
- D) Nothing; it just waits

Q189. [Concept] Which is correct?

- A) sleep() releases the lock; wait() does not
- B) wait() releases the monitor; sleep() does not
- C) Both release the lock
- D) Neither releases the lock

Q190. [Concept] Which statement is TRUE?

- A) setPriority() guarantees higher-priority threads run first
- B) yield() guarantees another thread will run
- C) join() makes the calling thread wait until the target thread finishes
- D) sleep() releases all locks

Q191. [Concept] `new ConcurrentHashMap<String,Integer>().put(null, 1)`:

- A) Works
- B) Throws NullPointerException
- C) Throws ConcurrentModificationException
- D) Compile-time error

Q192. [Concept] To invoke a private method `secret()` of another class using reflection, you need:

- A) `getMethod("secret")` and `invoke`
- B) `getDeclaredMethod("secret")`, `setAccessible(true)`, then `invoke`
- C) `getMethod("secret")` and `setAccessible(true)`
- D) It is impossible

Answer Key – Section 11

183. **B** – run() is a normal method call on the main thread; start() creates a new thread.
184. **B** – A thread can be started only once.
185. **A** – join() waits for T to finish.
186. **B** – count++ is not atomic (read-modify-write); use synchronized or AtomicInteger.
187. **B** – Instance locks belong to each object; a static synchronized method locks on the Class object.
188. **B** – wait/notify require owning the object's monitor.
189. **B** – wait() releases the monitor and sleep() keeps it.
190. **C** – Priorities and yield are only hints.
191. **B** – ConcurrentHashMap disallows null keys and values.
192. **B** – getMethod only finds public methods; getDeclaredMethod finds private ones.

Section 12: JDBC and DAO Pattern (Q193–Q200)

Q193. [Scenario] A login query is built as `"SELECT * FROM users WHERE name='" + n + "' AND pwd='" + p + "'"`. An attacker enters `' OR '1'='1` as the password. The best fix is:

- A) Keep Statement but remove spaces
- B) Use PreparedStatement with ? placeholders
- C) Use executeUpdate instead
- D) Convert the password to uppercase

Q194. [Trap]

```
PreparedStatement ps = con.prepareStatement("SELECT * FROM emp WHERE id =
?");
ps.setInt(0, 10);
```

- A) Works
- B) SQLException: parameter index out of range (indexes start at 1)
- C) Compile-time error
- D) NullPointerException

Q195. [Trap]

```
ResultSet rs = ps.executeQuery();
int id = rs.getInt(1);   // next() was never called
```

- A) Returns the first row's id
- B) SQLException: cursor is before the first row
- C) Returns 0
- D) Returns null

Q196. [Concept] To find the number and names of the columns returned by a query at runtime, use:

- A) DatabaseMetaData
- B) ResultSetMetaData
- C) ParameterMetaData
- D) Statement

Q197. [Scenario] A fund transfer debits account A (succeeds) and then the credit to account B fails with an SQLException. Auto-commit is left at its default. What is the result?

- A) Both operations are rolled back automatically
- B) The debit stays committed and the data is inconsistent
- C) Both operations are committed
- D) The JVM aborts the transaction

Q198. [Concept] In JDBC 4.0+ with the driver JAR on the classpath:

- A) Class.forName(driverClass) is always mandatory
- B) Explicit driver loading is not needed; DriverManager finds it via the service loader
- C) The driver can never be auto-loaded
- D) Only PreparedStatement needs Class.forName

Q199. [Concept] Which interface is used to call a stored procedure with OUT parameters?

- A) Statement
- B) PreparedStatement
- C) CallableStatement
- D) ResultSetMetaData

Q200. [Concept] The Data Access Object (DAO) pattern is used to:

- A) Separate persistence (JDBC) code from business logic
- B) Speed up SQL queries automatically
- C) Pool database connections
- D) Generate SQL automatically

Answer Key – Section 12

193. **B** – Placeholders treat input as data, not SQL.
194. **B** – JDBC parameter indexes start at 1.
195. **B** – The cursor starts before the first row; call rs.next() first.
196. **B** – ResultSetMetaData describes columns; DatabaseMetaData describes the database.
197. **B** – With auto-commit on, each statement commits immediately; use setAutoCommit(false), commit(), and rollback() in catch.
198. **B** – Service-loader discovery since JDBC 4.0.
199. **C** – CallableStatement supports registerOutParameter.
200. **A** – DAO encapsulates database access behind an interface.

Last-minute strategy for MCQ exams

- Most-failed topics: String pool and ==, Integer cache (-128 to 127), overloading vs overriding, field hiding, static vs instance, finally and return, ConcurrentModificationException, HashMap/HashSet equals-hashCode behavior, stream laziness, wildcard (? extends / ? super) rules.
- For output questions, first check for compile errors (missing cast, unreachable code, uncaught checked exceptions) before tracing values.
- Score yourself by section: below 70% in a section means re-read that session's notes and redo the questions after two days.